Tunnijuhend: söökla backend

11IT · Veebirakendused · seitsmes tund · tunnis

Viiendas tunnis tegid söökla arvustuste frontendi. Arvustused olid failis `andmed.ts`, mis tegi näo, et on backend. Täna teed päris backendi: Node'i programmi, mis annab samad arvustused JSON-ina. Reacti ühendame sellega järgmises tunnis.

> **Kuidas töötada.** Nagu eelmisel korral: kirjuta kood ise ümber, ära kopeeri, ja loe seletus kõrvalt läbi. Üks asi on uus. **Backendi vead ei ilmu brauseri konsooli, vaid terminali**, kus server töötab. Kui midagi ei tööta, vaata kõigepealt sinna. Konspekti lõpus on tabel levinumate vigadega.

Samm 0: ettevalmistus

Ava söökla projekt VS Code'is (File → Open Folder). Kui tegid terminali tunniülesannete 4. osa, on projektis juba kaust `server` ja selle sees `package.json`. Ava terminal ja kontrolli:

```
cd server
ls
```

Kui `ls` näitab `package.json`-it, on kõik korras. Kui kausta `server` pole, tee see nüüd (projekti juurkaustas):

```
mkdir server
cd server
npm init -y
```

Kontrolli ka Node'i versiooni käsuga `node -v`. Vaja on vähemalt `v20`. Kui on vanem, ütle õpetajale.

Samm 1: Express

Terminal peab olema kaustas `server` (`pwd` näitab lõpus `\server`). Siis:

```
npm install express
```

Vaata `ls`-iga, mis tekkis. Kaust `node_modules` ja fail `package-lock.json`. Ava `package.json` (`code package.json`) ja leia lõpust `dependencies`. Seal on nüüd Express.

> **Proovi:** mine `cd ..`-ga projekti juurkausta ja kirjuta `git status`. Kas `server/node_modules` on muutunud failide seas? Ei ole. Vite pani projekti tegemisel juurkausta faili `.gitignore` ja seal on rida `node_modules`. See kehtib kõigile alamkaustadele. Mine `cd server`-ga tagasi.

Samm 2: package.json korda

Kaks muudatust. Esiteks peab Node teadma, et kasutame `import`-i nagu Reactis. Teiseks tahame käsku `npm run dev`, mis paneb serveri käima ja taaskäivitab selle iga salvestuse peale. Muuda `server/package.json` selliseks:

`server/package.json`

```
{
  "name": "server",
  "version": "1.0.0",
  "type": "module",
  "main": "index.js",
  "scripts": {
    "dev": "node --watch index.js"
  },
  "dependencies": {
    "express": "^5.3.0"
  }
}
```

Kood	Mida see teeb
`"type": "module",`	Lubab `import`-i. Kui failis on juba `"type": "commonjs"`, muuda see `"module"`-iks. Kui sellist rida pole, lisa see.
`"scripts": { "dev": "node --watch index.js" }`	Rea `"test": …` asemele tuleb `"dev"`. `--watch` taaskäivitab serveri, kui salvestad.
`"express": "^5.3.0"`	Selle kirjutas npm. Ära muuda. Versioon võib sul olla teine.

Kui sinu failis on veel ridu (`description`, `author`, `license`), võivad need jääda.

> **JSON on range.** Iga rea lõpus on koma, välja arvatud ploki viimasel real enne `}`-sulgu. Üks puuduv või üleliigne koma ja `npm run dev` ütleb `JSON.parse`-i kohta midagi segast. Siis loe komad üle.

Samm 3: esimene server

Tee kausta `server` fail `index.js` (`ni index.js`, siis `code index.js`) ja kirjuta sinna:

`server/index.js`

```
import express from 'express';

const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.listen(PORT, () => {
  console.log(`Server töötab: http://localhost:${PORT}`);
});
```

Kood	Mida see teeb
`import express from 'express';`	Toob Expressi paketist.
`const app = express();`	Teeb serveri. Kõik reeglid paneme `app`-ile.
`const PORT = 3000;`	Port, millel server kuulab. Vite on 5173, meie võtame teise.
`app.get('/', (req, res) => {`	Reegel: GET-päring aadressile `/` käivitab selle funktsiooni. `req` on päring, `res` vastus.

Kood	Mida see teeb
`res.send('Tere! ...');`	Saadab vastuseks teksti.

`app.listen(PORT, () => {`
`  console.log(...);`
`});`
	Paneb serveri käima ja ootama. Kui server on valmis, kirjutab terminali aadressi. Tagurpidi ülakomad `` `…` `` ja `${PORT}` panevad muutuja teksti sisse, nagu Pythoni f-string.

Pane server käima. Terminal peab ikka olema kaustas `server`:

```
npm run dev
```

Terminal ütleb `Server töötab: http://localhost:3000`. Ctrl-klõpsa aadressil või kirjuta see brauserisse. Ekraanil on sinu tekst.

> **Proovi:** muuda tekst `'Tere! …'` millekski muuks ja salvesta. Mis juhtus terminalis? Värskenda brauserit. Siis vajuta terminalis `Ctrl+C` ja värskenda brauserit uuesti. Mida brauser nüüd ütleb ja miks? Pane server `npm run dev`-ga tagasi käima.

Samm 4: arvustused JSON-ina

Nüüd paneme serverisse samad arvustused, mis on frontendis failis `andmed.ts`, ja teeme neile aadressi. Lisa `index.js`-i `const PORT` rea alla massiiv ja esimese marsruudi alla teine marsruut:

`server/index.js, const PORT rea alla`

```
const arvustused = [
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },
];
```

`server/index.js, app.get('/') marsruudi alla`

```
app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});
```

Kood	Mida see teeb
`const arvustused = [ … ];`	Täpselt sama massiiv, mis `andmed.ts`-is, ainult tüübita, sest see fail on JavaScript, mitte TypeScript. Andmed elavad nüüd serveri mälus.
`app.get('/api/arvustused',`	Uus reegel uuele aadressile. `/api` alguses on kokkulepe: kõik, mis annab andmeid, algab nii. Nii on kohe näha, mis on leht ja mis on andmed.
`res.json(arvustused);`	Saadab terve massiivi JSON-ina.

Salvesta. Server taaskäivitub ise. Ava `http://localhost:3000/api/arvustused`. Kas see meenutab midagi? Täpselt nii nägi välja PokeAPI vastus. Ainult et nüüd oled sina see, kes vastab.

Ava F12 → **Network** ja värskenda lehte. Klõpsa päringul `arvustused`. Leia staatusekood (peaks olema 200) ja `Content-Type`. Selle päise pani `res.json` ise.

> **Proovi:** kirjuta aadressiks `/api/arvustus` (lõpust puudub `-ed`). Mida leht ütleb? Kes selle teate kirjutas? Sina igatahes mitte.

Samm 5: üks arvustus

Päris API annab ka ühe asja korraga, näiteks `/api/arvustused/2`. Lisa `index.js`-i eelmise marsruudi alla:

`server/index.js`

```
app.get('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const arvustus = arvustused.find((a) => a.id === id);
  if (!arvustus) {
    return res.status(404).json({ viga: 'Sellist arvustust ei ole' });
  }
  res.json(arvustus);
});
```

Kood	Mida see teeb
`'/api/arvustused/:id'`	Koolon teeb `id`-st muutuja. See reegel sobib aadressidele `/api/arvustused/1`, `/2`, `/abc` ja nii edasi.
`const id = Number(req.params.id);`	`req.params.id` on see, mis aadressis koolonikohal oli. Aadress on alati tekst, nii et `'2'`. `Number()` teeb sellest arvu 2.
`arvustused.find((a) => a.id === id)`	`find` käib massiivi läbi ja annab esimese, mille kohta tingimus on tõene. Kui ühtegi pole, annab `undefined`.
`if (!arvustus) {`	`!` on „ei“. Kui arvustust ei leitud…
`return res.status(404).json({ viga: … });`	…saadame 404 ja selgituse. `return` on oluline: see lõpetab funktsiooni, et kood ei jõuaks allolevale `res.json`-ile. Ühele päringule saab vastata ainult üks kord.
`res.json(arvustus);`	Leiti, saadame selle ühe arvustuse.

Proovi brauseris `/api/arvustused/2`, `/api/arvustused/7` ja `/api/arvustused/abc`. Mis staatusekoodi näitab Network? Mis on `Number('abc')` ja miks läheb ka see 404-sse?

> **Proovi:** kustuta `Number(` ja `)`, nii et rida on `const id = req.params.id;`. Ava `/api/arvustused/2`. Arvustus 2 on ju olemas, miks siis 404? Vihje: `===` võrdleb ka tüüpi. Pane `Number` tagasi.

Samm 6: salvesta Gitti

Mine projekti juurkausta ja tee commit:

```
cd ..
git status
git add .
git commit -m "Esimene backend: arvustused JSON-ina"
```

`git status` peab lõpuks ütlema `nothing to commit`. Ja `node_modules`-it seal ei olnud, nagu samm 1 lubas.

Kogu fail

Kui midagi ei tööta, võrdle oma faili sellega. Ära kopeeri seda sisse, otsi, kus on erinevus.

`server/index.js`

```
import express from 'express';

const app = express();
const PORT = 3000;

const arvustused = [
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },
];

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});

app.get('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const arvustus = arvustused.find((a) => a.id === id);
  if (!arvustus) {
    return res.status(404).json({ viga: 'Sellist arvustust ei ole' });
  }
  res.json(arvustus);
});

app.listen(PORT, () => {
  console.log(`Server töötab: http://localhost:${PORT}`);
});
```

Kui jõuad varem valmis

Need on väljakutsed: koodi pole, ainult vihjed. Iga väljakutse on uus marsruut `index.js`-is. Proovi brauseris järele.

- **Statistika.** `GET /api/statistika` vastab `{ "arv": 3, "keskmine": 4 }`. Vihje: `arvustused.length` ja `for`-tsükkel, mis liidab hinded kokku.
- **Ainult head.** `GET /api/arvustused?min=4` annab ainult need, mille hinne on vähemalt 4. Vihje: `req.query.min` ja `.filter()`. Mõtle, mis peab juhtuma, kui `?min` aadressis üldse puudub. Seda tuleb muuta olemasolevas marsruudis, mitte uues.
- **Menüü.** `GET /api/toidud` annab ainult toitude nimed: `["Kanakaste riisiga", "Hernesupp", …]`. Vihje: `.map()`.
- **Soovitus.** `GET /api/soovitus` annab ühe juhusliku arvustuse, mille hinne on vähemalt 4. Iga värskendusega võib tulla erinev. Vihje: `Math.random()` ja `Math.floor()`.

> **Hoia projekt alles!** Järgmises tunnis paneme kaks serverit omavahel rääkima: React küsib arvustused sinu backendilt ja `andmed.ts` läheb prügikasti.